Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země

Jan Bartůšek, Petr Havel

Aplikace počítačové kartografie

Generalizace budov LOD0

1. Zadání

Vstup: množina budov $B = \{B_i\}_{i=}^{n}$, *budova* $B_i = \{P_{i,j}\}_{j=1}^{m}$.

Výstup: $G(B_i)$.

Ze souboru načtěte vstupní data představovaná lomovými body budov a proveďte generalizaci budov do úrovně detailu LOD0. Pro tyto účely použijte vhodnou datovou sadu, např. ZABAGED, testování proveďte nad třemi datovými sadami (historické centrum města, sídliště, izolovaná zástavba).

Pro každou budovu určete její hlavní směry metodami:

- Minimum Area Enclosing Rectangle,
- PCA.

U první metody použijte některý z algoritmů pro konstrukci konvexní obálky. Budovu při generalizaci do úrovně LOD0 nahraďte obdélníkem orientovaným v obou hlavních směrech, se středem v těžišti budovy, jeho plocha bude stejná jako plocha budovy. Výsledky generalizace vhodně vizualizujte.

Otestujte a porovnejte efektivitu obou metod s využitím hodnotících kritérií. Pokuste se rozhodnout, pro které tvary budov dávají metody nevhodné výsledky, a pro které naopak poskytují vhodnou aproximaci.

Hodnocení:

Krok	**Hodnocení**
Generalizace budov metodami Minimum Area Enclosing Rectangle a PCA.	15b
Generalizace budov metodou Longest Edge.	*+5b*
Generalizace budov metodou Wall Average.	*+8b*
Generalizace budov metodou Weighted Bisector.	*+10b*
Implementace další metody konstrukce konvexní obálky.	*+5b*
Ošetření singulárních případů při generování konvexní obálky.	*+2b*
*Načtení vstupních dat ze *.shp.*	*+10b*
Max celkem:	**55b**

2. Údaje o bonusových úlohách + počet bodů

Nad rámec povinného základu byla aplikace rozšířena o následující bonusové funkcionality (potenciál pro zisk maximálního počtu bodů):

Bonusové metody generalizace: Implementace metod *Longest Edge*, *Wall Average* a *Weighted Bisector* pro lepší zpracování komplexních tvarů (L, Z).

Pokročilá konvexní obálka: Vedle metody Jarvis Scan byla naimplementována metoda *Graham Scan* s optimalizovanou časovou složitostí $O(n \log n)$.

Nativní práce s GIS daty: Plná podpora formátu ESRI Shapefile pomocí knihovny GeoPandas, zahrnující dekompozici geometrií typu MultiPolygon.

Analytický a statistický modul: Aplikace měří časovou efektivitu algoritmů a přesnou míru vyhlazení reprezentovanou procentuálním zmenšením obvodu budov.

Celkový možný počet bodů: 55

3. Popis a rozbor problému a vzorce

Ústředním problémem úlohy je nalezení tzv. hlavního směru budovy, který definuje natočení výsledného generalizovaného obdélníku.

Minimum Area Enclosing Rectangle (MAER) hledá obdélník s nejmenší možnou plochou opsaný vstupní množině bodů. Platí, že nejméně jedna strana obdélníku s minimální plochou je kolineární s některou ze stran konvexní obálky $H(P)$. Krokem předzpracování je tedy nalezení $H(P)$, jejíž hrany tvoří množinu testovaných úhlů natočení $-\sigma$.

Principal Component Analysis (PCA) Statistická metoda využívající singulární rozklad (SVD) kovarianční matice vrcholů budovy. Kovarianční matice C je dána vztahem:

$$C(A,B) = \frac{1}{n-1}\sum_{i=1}^{n}(A_i - \mu_A)(B_i - \mu_B)$$

Singulární rozklad matice $C = U\Sigma V^T$ poskytne vlastní $vektory$ ($matice\, V$), přičemž první hlavní komponenta udává směr s nejvyšším rozptylem bodů, tedy hlavní směr budovy.

Wall Average a Weighted Bisector Metody založené na váženém průměru směrnic jednotlivých stěn budovy. U Wall Average je aplikována modulární aritmetika s násobkem úhlu 4α, aby se na sebe kolmé stěny (svírající úhel $\pi/2$ ztotožnily do jednoho směru. U metody Weighted Bisector je aplikován násobek 2α, což identifikuje osu symetrie mezi paralelními stěnami. Hlavní směr je dán váženým průměrem:

$$\sigma = \frac{s_1\sigma_1 + s_2\sigma_2}{s_1 + s_2}$$

Zachování plochy (Area Preservation) Plocha původního zjednodušeného obdélníku A musí být proporcionálně transformována na plochu původní budovy A_b. Je vypočten plošný koeficient k:

$$k = \frac{A_b}{A}$$

Protože plocha roste s druhou mocninou délky, lineární rozměry vektorů od těžiště T ke každému vrcholu V_i se musí přeškálovat o $\sqrt{k}$. Nové souřadnice generalizovaného obdélníku jsou dány vztahem:

$$V_i' = T + \sqrt{k}u_i$$

4. Popisy algoritmů formálním jazykem

Algoritmus: Graham Scan pro konvexní obálku 1: Nalezení pivota $q = \min(y_i)$, $q \in H$ 2: Setřídění $\forall\, p_i \in S$ dle polárního úhlu $\omega_i = \angle(p_i, q, x)$ 3: Inicializace zásobníku: $S \leftarrow q, S \leftarrow p_1$ 4: Pro všechny zbývající setříděné body p_j: 5: Zatímco p_j netvoří levotočivou zatáčku s posledními dvěma body v S: 6: S.pop() (odstranění reflexního vrcholu) 7: S.push(p_j) 8: Vrať prvky v zásobníku S jako konvexní obálku H.

Algoritmus: Minimum Area Enclosing Rectangle (MAER) 1: Najdi H = CH(P) 2: Inicializuj minimální plochu $\underline{A} = \infty$ a obdélník R 3: Opakuj pro každou hranu $e \in H$: 4: Spočti směrnici \sigma hrany e 5: Otoč body P o úhel -\sigma: P_0 = R($-\sigma$)P 6: Najdi Bounding Box pro P_0 *a urči jeho plochu A* 7: Pokud A < $\underline{A}$ 8: $\underline{A}$ = A, $\underline{\sigma} = \sigma$, ulož aktuální Bounding Box 9: Zrotuj uložený Bounding Box zpět o úhel $\underline{\sigma}$ do geografické polohy.

5. Problematické situace a jejich rozbor (Simplexy)

Během vývoje a testování nad produkčními GIS daty bylo nutné ošetřit několik kritických geometrických singularit.

A) Dělení nulou při výpočtu úhlu vektorů

Reálná data (např. z OpenStreetMap) často obsahují duplicitní body na stejných souřadnicích (zero-length edge). Výpočet normy vektoru by vedl k nule a následně k chybě ZeroDivisionError.

Řešení v kódu (soubor algorithms.py, metoda get2VectorsAngle):

```
# Norms
nu = math.sqrt(ux**2 + uy**2)
nv = math.sqrt(vx**2 + vy**2)

# Prevent division by zero for identical points
if nu == 0 or nv == 0:
    return 0.0
```

B) Kolaps geometrie na přímku U velmi úzkých nebo poškozených budov může Bounding Box zkolabovat do úsečky. Výpočet plošného koeficientu k by způsobil dělení nulou. *Řešení v kódu (soubor algorithms.py, metoda resizeRectangle):*

```
A = self.getArea(mbr)

# Prevent division by zero if geometry collapses to a line
if A == 0:
    return mbr

Ab = self.getArea(building)
k = Ab / A
```

C) Počáteční vektor u metody Jarvis Scan U budov s dokonale vodorovnou spodní stěnou může vyhledání levého dolního pivota vést k selhání výpočtu počátečního úhlu, pokud bychom jako referenční bod použili bod ležící na budově. *Řešení v kódu (soubor algorithms.py, metoda createCH):*

```
# Initial segment (force a horizontal vector pointing strictly to the left)
pj = q
pj1 = QPointF(q.x() - 1.0, q.y())
```

6. Vstupní data, formát vstupních dat, popis

Aplikace akceptuje standardizovaný vektorový formát ESRI Shapefile (.shp). Načítání dat využívá robustní prostorovou knihovnu geopandas, která geometrie transformuje na iterovatelné objekty typu Polygon nebo MultiPolygon a extrahuje souřadnice vnějších prstenců (exterior rings) pro účely LOD0

generalizace. V souladu s požadavky zadání proběhlo testování a evaluace algoritmů nad třemi odlišnými datovými sadami. Data byla získána z databáze OpenStreetMap (pomocí nástroje Overpass Turbo) a reprezentují tři základní urbanistické struktury:

Historické centrum města (Centrum Prahy): Zástavba charakteristická vysokou hustotou, nepravidelnými, zkosenými a vysoce členitými půdorysy, které historicky kopírují úzké a křivolaké uliční sítě.

Sídliště (Praha – Chodov): Rozlehlá bloková a panelová zástavba. Typické jsou obří, často ortogonální objekty (tvary písmene L, U nebo Z). Domy mají sice převážně pravé úhly, ale jejich výrazně nekonvexní tvar představuje výzvu pro některé aproximační generalizační metody.

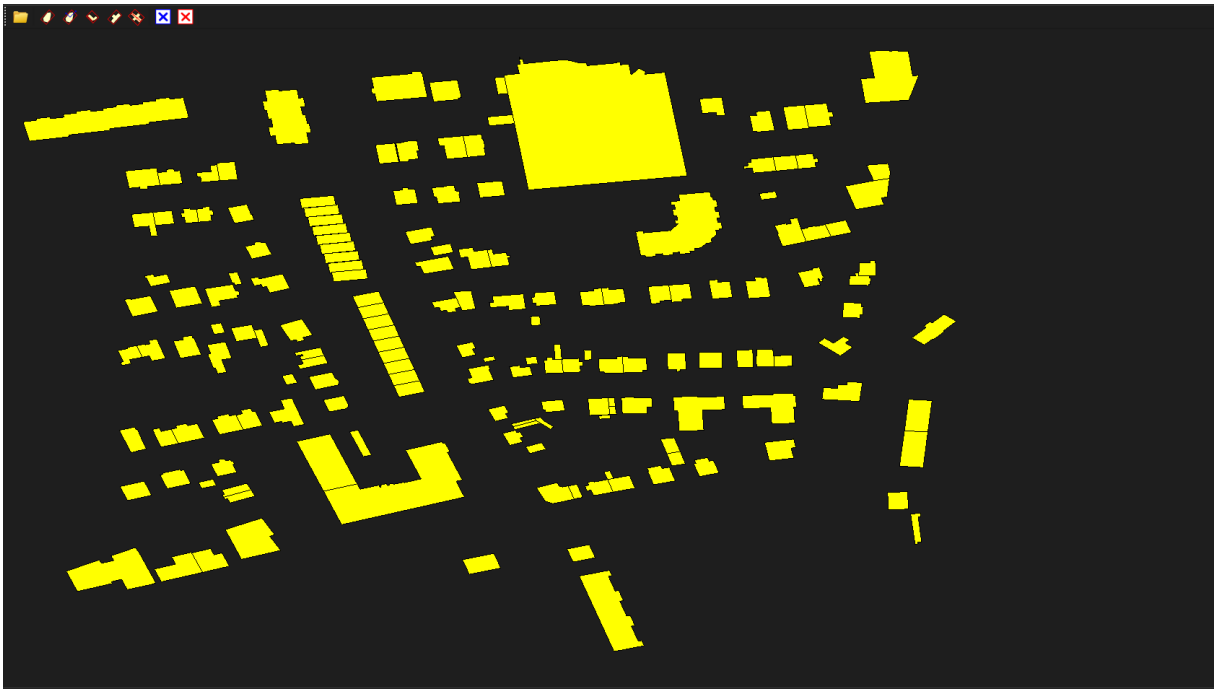
Izolovaná zástavba (Obec u Domažlic): Menší venkovské sídlo. Typické jsou solitérní rodinné domy či menší zemědělské objekty s jednoduššími, převážně obdélníkovými půdorysy, s různým prostorovým natočením a dostatečnými rozestupy.

Výstupem aplikace je vizuální a statistická komparace nad vstupními daty.

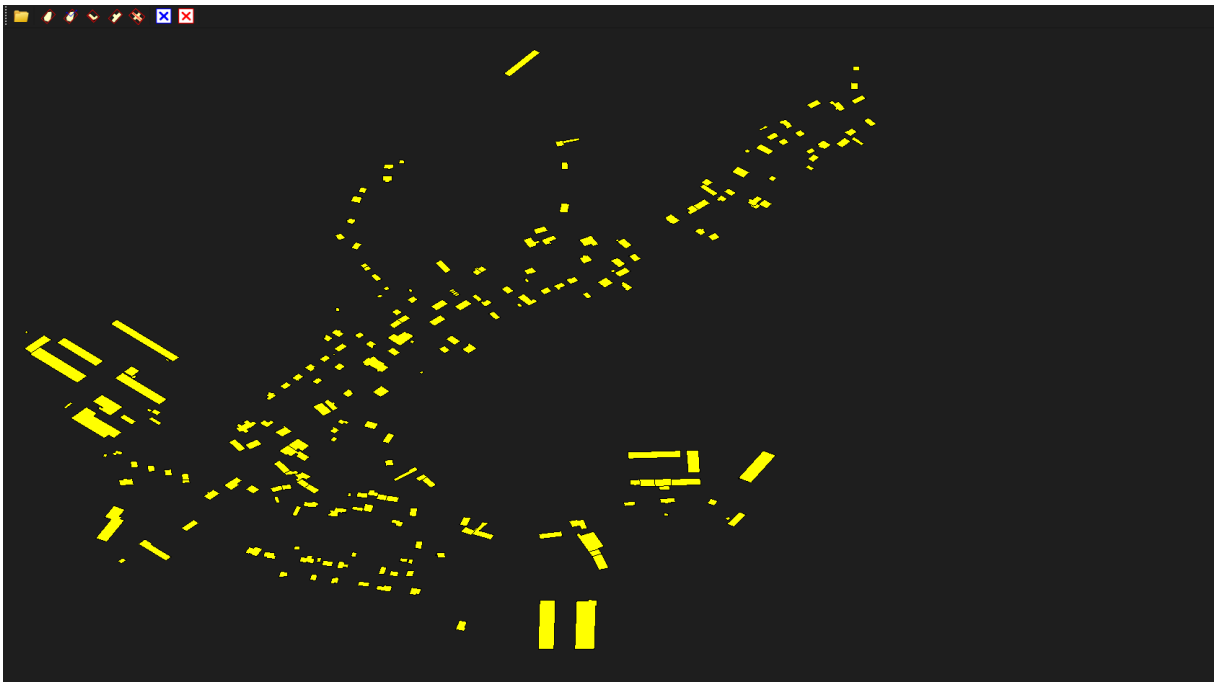
Vizuální výstup: Uživatelské plátno (Canvas dědící z QWidget) provádí vykreslení pomocí třídy QPainter. Původní členité budovy jsou plošně vyplněny, generalizované tvary jsou overlayovány jako prázdné červené obdélníky pro vizuální verifikaci křížení.

Kvantitativní výstup: Při dokončení výpočtu se uživateli zobrazí agregovaný statistický log ve formátu QMessageBox. Log obsahuje název metody, celkový výpočetní čas (v sekundách přes modul time), průměrný původní a generalizovaný obvod a exaktní metriku procentuálního vyhlazení.

Obr. 1: Historické centrum (Praha). Ukázka aplikace metody Weighted Bisector nad nepravidelnými a hustě naskládanými polygeny historické zástavby.



Obr. 2: Sídliště (Praha - Chodov). Aplikace metody Wall Average na ortogonální blokovou zástavbu. Ukázka úspěšného nalezení hlavního směru a zachování orientace u budov složitých tvarů (L, U).



Obr. 3: Izolovaná zástavba (obec u Domažlic). Aplikace metody MAER (Minimum Area Enclosing Rectangle) nad solitérními objekty venkovského charakteru, včetně zobrazení statistického okna s výsledky generalizace.

7. Výstupní data, formát výstupních dat, popis

Výstupem aplikace je vizuální a statistická komparace nad vstupními daty.

Vizuální výstup: Uživatelské plátno (Canvas dědící z QWidget) provádí vykreslení pomocí třídy QPainter. Původní členité budovy jsou plošně vyplněny, generalizované tvary jsou overlayovány jako prázdné červené obdélníky pro vizuální verifikaci křížení.

Kvantitativní výstup: Při dokončení výpočtu se uživateli zobrazí agregovaný statistický log ve formátu QMessageBox. Log obsahuje název metody, celkový výpočetní čas (v sekundách přes modul time), průměrný původní a generalizovaný obvod a exaktní metriku procentuálního vyhlazení.

8. Dokumentace: Popis tříd, datových položek a klíčových metod

Aplikace je navržena plně objektově a striktně odděluje grafické uživatelské rozhraní od výpočetní logiky. Stěžejní částí softwaru je třída Algorithms (soubor algorithms.py), která neudržuje žádný vnitřní stav, ale zapouzdřuje veškerou matematickou a geometrickou logiku generalizace.

Níže je uveden detailní rozbor klíčových metod doplněný o ukázky zdrojového kódu a vysvětlení jejich algoritmické podstaty.

8.1. Metoda pro zachování plochy (resizeRectangle)

Základním kartografickým požadavkem LOD0 generalizace je zachování původní rozlohy budovy. Tato metoda zajišťuje proporcionální zmenšení či zvětšení vypočteného obdélníku z jeho těžiště tak, aby se jeho výsledná plocha (A) exaktně rovnala ploše původní budovy (A_b).

```python
def resizeRectangle(self, building: QPolygonF, mbr: QPolygonF):
    A = self.getArea(mbr)

    # Prevent division by zero if the geometry collapses to a straight line (a
    if A == 0:
        return mbr

    Ab = self.getArea(building)

    # Calculate the area ratio 'k'
    k = Ab / A

    # Compute the geometric center (centroid) of the rectangle
    x_c = (mbr[0].x() + mbr[1].x() + mbr[2].x() + mbr[3].x()) / 4
    y_c = (mbr[0].y() + mbr[1].y() + mbr[2].y() + mbr[3].y()) / 4

    mbr_res = QPolygonF()
    for i in range(4):
        # Create a vector from the centroid to the rectangle's vertex
        v_x = mbr[i].x() - x_c
        v_y = mbr[i].y() - y_c

        # Scale the vector. Since area scales quadratically (A = w*h), linear
        # must be scaled by the square root of the area ratio (sqrt(k)).
        v_x_res = v_x * math.sqrt(k)
        v_y_res = v_y * math.sqrt(k)

        # Translate the scaled vector back to global coordinates
        p_x = v_x_res + x_c
        p_y = v_y_res + y_c

        mbr_res.append(QPointF(p_x, p_y))

    return mbr_res
```

Algoritmus nejprve spočítá plošný koeficient k. Vzhledem k tomu, že plocha roste s druhou mocninou lineárních rozměrů, vektory směřující z těžiště ke čtyřem vrcholům obdélníku musí být přenásobeny hodnotou $\sqrt{k}$. Transformované vrcholy jsou následně vráceny zpět do globálního souřadnicového systému.

8.2. Konstrukce konvexní obálky – Graham Scan (createCHGrahamScan)

Pro optimalizaci výpočtu MAER byl implementován algoritmus Grahamova skenu s časovou složitostí $O(n\ log\ n)$, který je na komplexních geometriích budov výrazně efektivnější než standardní Jarvisův pochod.

```python
def createCHGrahamScan(self, pol: QPolygonF) -> QPolygonF:
    # Create Convex Hull using the Graham Scan algorithm. Time complexity: O(n lo
    # Highly efficient for complex building geometries.
    if len(pol) < 3:
        return pol

    # Step 1: Find the lowest point (pivot). If tied, pick the left-most one.
    pivot = min(pol, key=lambda p: (p.y(), p.x()))

    # Step 2: Extract all unique points to avoid sorting errors and filter out th
    points = []
    for p in pol:
        if p != pivot:
            if not any(p.x() == pt.x() and p.y() == pt.y() for pt in points):
                points.append(p)

    # Step 3: Sort remaining points strictly by their polar angle with the pivot.
    points.sort(key=lambda p: self.getGrahamAngleAndDist(p, pivot))

    # Step 4: Initialize the state stack with our starting pivot
    stack = [pivot]

    # Step 5: Process points. Maintain convexity by popping points that create a
    for p in points:
        while len(stack) > 1:
            p1 = stack[-2]
            p2 = stack[-1]
            p3 = p

            # Check the orientation of the last three points
            cp = self.getOrientation(p1, p2, p3)

            # If cross product <= 0, the segment forms a concave dent or straight
            # Pop the middle point to maintain strict convexity.
            if cp <= 0:
                stack.pop()
            else:
                break
        stack.append(p)

    # Step 6: Convert the optimized stack back into a QPolygonF structure
    ch = QPolygonF()
    for p in stack:
        ch.append(p)

    return ch
```

Algoritmus využívá datovou strukturu zásobník (stack). V každém kroku se testují poslední tři body pomocí křížového součinu 2D vektorů (metoda getOrientation). Pokud tyto tři body netvoří striktní levotočivou zatáčku (křížový součin ≤ 0), prostřední bod je ze zásobníku nevratně odstraněn (stack.pop()). To garantuje, že výsledný polygon bude ryze konvexní.

8.3. Minimum Area Enclosing Rectangle (createMBR)

Metoda MAER iteruje přes všechny hrany konvexní obálky. Využívá matematického teorému, který dokazuje, že obdélník s minimální plochou opsaný polygonu musí mít alespoň jednu hranu kolineární s hranou jeho konvexní obálky.

```python
def createMBR(self, building: QPolygonF):
    # Calculates the Minimum Area Enclosing Rectangle (MAER)
    sigma_min = 0

    # Step 1: Pre-compute the convex hull to drastically reduce the numbe
    ch = self.createCHGrahamScan(building)

    # Initialize with an unrotated Axis-Aligned Bounding Box
    mmb_min, area_min = self.createMMB(ch)

    # Step 2: Iterate through every edge of the convex hull
    n = len(ch)
    for i in range(n):
        dx = ch[(i+1)%n].x() - ch[i].x()
        dy = ch[(i+1)%n].y() - ch[i].y()

        # Find the orientation angle of the current edge
        sigma = math.atan2(dy, dx)

        # Rotate the entire hull to align this specific edge horizontally
        ch_r = self.rotatePolygon(ch, -sigma)

        # Calculate the bounding box for this alignment
        mmb, area = self.createMMB(ch_r)

        # If this alignment yields a smaller area, save it as the new min
        if area < area_min:
            area_min = area
            mmb_min = mmb
            sigma_min = sigma

    # Step 3: Rotate the smallest found box back to its original geograph
    return self.rotatePolygon(mmb_min, sigma_min)
```

Pre-procesingem budovy na konvexní obálku se drasticky snižuje počet hran nutných k testování. Celý polygon obálky se zrotuje tak, aby byla aktuálně testovaná hrana vodorovná s osou X (úhel $-\sigma$). Následně se spočte standardní pravoúhlý min-max box a jeho plocha. Zpětnou rotací o uložený úhel σ_{min} se získá finální MBR.

8.4. Principal Component Analysis (simplifyBuildingPCA)

Statistická metoda hledající osy maximálního prostorového rozptylu bodů geometrie pomocí singulárního rozkladu matic (SVD).

```
def simplifyBuildingPCA(self, building: QPolygonF):
    # Generalize using Principal Component Analysis (PCA)
    # Finds the direction of maximum spatial variance of the building's verti
    X, Y = [], []

    # Extract X and Y coordinates into statistical arrays
    for p in building:
        X.append(p.x())
        Y.append(p.y())

    A = np.array([X, Y])

    # Compute the 2x2 Covariance Matrix to understand coordinate correlation
    C = np.cov(A)

    # Perform Singular Value Decomposition (SVD) to extract eigenvectors and e
    [U, S, V] = np2.svd(C)

    # The first principal component (direction of highest variance) is derive
    sigma = math.atan2(V[0][1], V[0][0])

    # Align the building along the principal component, calculate AABB, rotat
    build_rot = self.rotatePolygon(building, -sigma)
    mmb, area = self.createMMB(build_rot)
    mbr = self.rotatePolygon(mmb, sigma)

    return self.resizeRectangle(building, mbr)
```

Kovarianční matice (C) zachycuje vzájemnou prostorovou korelaci X a Y souřadnic lomových bodů. Aplikací SVD rozkladu nad touto maticí získáme vlastní vektory (matice V). První vlastní vektor ($V[0]$) vždy směřuje do osy největší variance bodů, a tím jednoznačně a velmi rychle definuje hlavní natočení budovy.

8.5. Vážený průměr stěn (simplifyBuildingWallAverage)

Pokročilá aproximační metoda řešící generalizaci nepravidelných ortogonálních budov (např. tvaru L či U), kde by prostý výpočet PCA nebo nejdelší hrany mohl selhat.

```
def simplifyBuildingWallAverage(self, building: QPolygonF) -> QPolygonF:
    # Generalization using the Weighted Wall Average method.
    # addresses the problem of irregular orthogonal buildings (L-shapes, U-s
    n = len(building)
    sum_x = 0.0
    sum_y = 0.0

    for i in range(n):
        p1 = building[i]
        p2 = building[(i + 1) % n]
        line = QLineF(p1, p2)
        length = line.length()

        # Skip invalid zero-length artifacts
        if length == 0:
            continue

        angle_rad = math.radians(line.angle())

        # Multiplying by 4 maps all these orthogonal angles to 360 (0) degre
        sum_x += length * math.cos(4 * angle_rad)
        sum_y += length * math.sin(4 * angle_rad)

    # Calculate the average vector angle and divide by 4 to revert the mathe
    avg_angle_rad = math.atan2(sum_y, sum_x) / 4.0
    avg_angle_deg = math.degrees(avg_angle_rad)

    # Apply the averaged orientation
    transform = QTransform()
    transform.rotate(avg_angle_deg)
    rotated_building = transform.map(building)

    # Generate bounding box in the averaged orientation
    bbox = rotated_building.boundingRect()
    bbox_poly = QPolygonF([
        bbox.topLeft(), bbox.topRight(),
        bbox.bottomRight(), bbox.bottomLeft()
    ])

    # Rotate back to geographic space and adjust final area
    inv_transform = QTransform()
    inv_transform.rotate(-avg_angle_deg)
    final_rect = inv_transform.map(bbox_poly)

    return self.resizeRectangle(building, final_rect)
```

Metoda využívá délku hrany (length) jako váhový koeficient. Nejdůležitějším matematickým mechanismem je přenásobení úhlu každé stěny čtyřmi. Tento trik způsobí, že všechny stěny tvořící kolmý systém budovy (např. svislé i vodorovné zdi) se úhlově ztotožní a začnou matematicky posilovat jeden společný dominantní směrnicový vektor (sum_x, sum_y). Výsledný úhel se před rotací opět vydělí čtyřmi.

9. Závěr, hodnocení nad datovými sadami a náměty na vylepšení

Cíle stanovené v zadání byly plně dosaženy, a to včetně implementace optimalizovaných konvexních obálek (Graham Scan) a tří bonusových generalizačních metod. Kvantitativní i vizuální testování nad třemi definovanými datovými sadami odhalilo jasná specifika a vhodnost jednotlivých algoritmů:

Sídliště (Chodov): Na ortogonálních objektech typu "L" nebo "Z" selhává metoda nejdelší hrany (Longest Edge), která tvar často nechtěně zkoulí. Metoda PCA vykazuje podobné defekty, neboť osa největší variance bodů nemusí odpovídat pravoúhlé povaze panelového domu. Absolutně nejlepších výsledků zde dosahuje metoda **Wall Average**, která díky modulárnímu násobení úhlů 4\alpha dokonale sjednotí kolmé stěny a zarovná výsledný obdélník s hlavní uliční čarou.

Historické centrum (Praha): Na vysoce nepravidelných a zkosených polygonech dominovaly metody **MAER** a **Weighted Bisector**. Metoda Weighted Bisector dokázala u historických budov velmi efektivně najít podélnou osu symetrie, a to i v případech, kdy budova neměla žádné zjevné pravé úhly.

Izolovaná zástavba (Domažlicko): U jednodušších, solitérních objektů (typické rodinné domy) poskytovaly všechny metody vizuálně velmi podobné a kvalitní výsledky. Z hlediska výpočetní efektivity se zde ukázala jako nejvhodnější metoda **PCA**, která je řádově rychlejší a na geometricky jednoduchých tvarech generuje zcela relevantní generalizaci.

Možná vylepšení: Současná podoba algoritmů generalizuje budovy čistě lokálně (izolovaně). V husté zástavbě (centrum Prahy) nebo na sídlištích tento lokální přístup generuje topologické konflikty – zmenšené obdélníky sousedních budov se začnou překrývat. Ideálním vylepšením by byla nadstavba pro řešení topologických relací (globální posun geometrií k odstranění překryvů) a integrace modulu pro export generalizovaných dat zpět do formátu .shp.

10. Seznam literatury

[1] BAYER, Tomáš. *Konvexní obálka množiny bodů, MAER* [Prezentace]. Praha: Katedra aplikované geoinformatiky a kartografie, Přírodovědecká fakulta Univerzity Karlovy.

[2] DUCHÊNE, C., et al. (2003). *Využití Weighted Bisector a Wall Average v detekci orientace*.

[3] QGIS.org. (2024). *Geopandas Documentation*. Dostupné z: geopandas.org

[4] Riverbank Computing. (2024). *PyQt6 Reference Guide*